

VIDCHOPPER ARCHITECTURE

Shared Engine and Qt Boundary Guide

Stable 1.0.0 shared-engine contract - Manager and developer reference

Source SHA-256 44f4ca09b7efca15448d07bcee5dfac8fbf6f196ad16f8eb81b432da1e163852

Stable 1.0.0 implementation contract | Linear VID-38 | Manager and developer reference

This guide is the authoritative contract for converging the GUI and CLI on one shared, Qt-free engine. It turns the decision in [ADR 0001](#) ([../knowledge/architecture/decisions/0001-shared-qt-free-engine.md](#)) into implementable interfaces and migration gates. Canonical product nouns come from the [domain glossary](#) ([../CONTEXT.md](#)); this guide does not redefine them.

Normative words (MUST, MUST NOT, SHOULD, and MAY) describe the stable-release boundary. Code blocks show the required public shape. An implementation task may split declarations across headers, but changing a name, state, or semantic below requires an ADR update before code changes.

Decision at a Glance

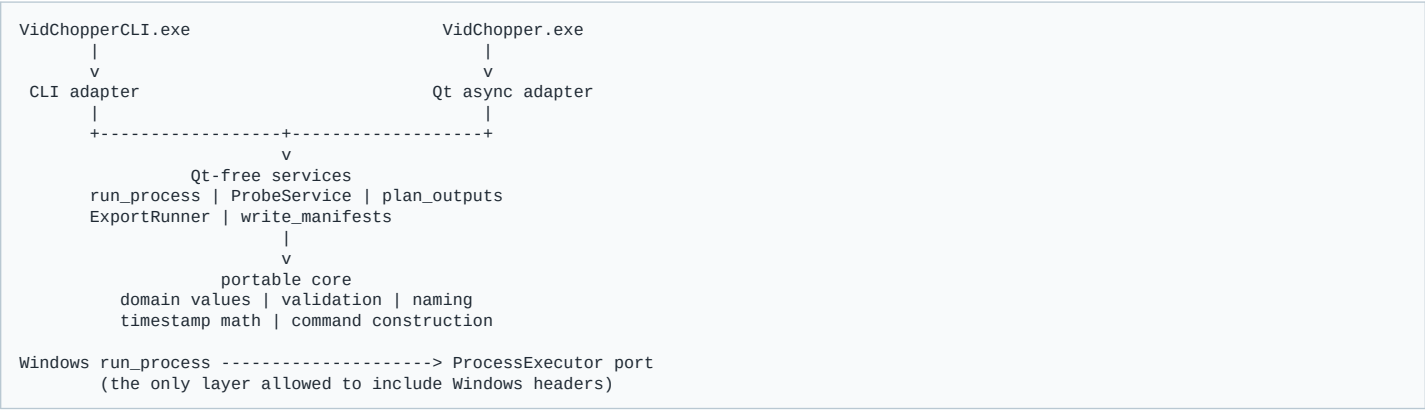
VidChopper will have one synchronous C++20 engine for ChapterSource resolution, probing, planning, export, manifests, cancellation, and progress. The CLI calls it directly. A thin Qt adapter runs the same engine on a worker thread and translates immutable events to queued Qt signals.

Concern	Owner	Stable rule
Domain values and pure validation	core	Standard C++ only; no Qt, processes, or filesystem writes.
Workflow services	services	Synchronous, Qt-free contracts shared by GUI and CLI.
Windows process execution	platform/windows	Win32 details stay behind the process-runner service.
CLI presentation	cli adapter	Parsing, text output, and exit-code mapping only.
Desktop presentation	qt adapter	QObject lifetime, worker scheduling, dialogs, and signals only.
Executables	composition roots	Construct concrete services and adapters; own no workflow rules.

The migration MUST preserve user-visible names, settings precedence, output paths, ffmpeg argument ordering, manifests, exit codes, and bounded progress unless a separate issue explicitly changes one of those compatibility surfaces.

Dependency Direction

An arrow means "may depend on." There are no sideways dependencies between adapters.



The allowed edges are:

From	May depend on	MUST NOT depend on
core	C++20 standard library	Qt, Win32, process APIs, presentation adapters
services	core and service ports	Qt, QObject, QString, QProcess, UI state
platform/windows	service ports, core, Win32	Qt or presentation policy
cli adapter	services, core	Qt and qt adapter
qt adapter	services, core, Qt	cli adapter or duplicated workflow
executables	adapters and concrete services	domain decisions or orchestration

The target build graph is `vidchopper_core <- vidchopper_services <- adapters <- executables`. `vidchopper_platform_windows` implements a services port and is selected only by composition roots. The shared libraries MUST build and run their fast tests without a Qt SDK.

Public C++20 Contracts

These declarations use existing project aliases such as `u8`, `i32`, and `Path`. All strings crossing the shared boundary are UTF-8. All paths are `std::filesystem::path` values. The migration reuses the concrete contracts already implemented under `src/services/` and `src/cli/`. It MUST NOT add a parallel result framework, custom cancellation token, or renamed process vocabulary merely to move ownership.

Process runner and cancellation

```
enum class ProcessExitState : u8 {
    Success = 0,
    FailedStart = 1,
    TimedOut = 2,
    Crashed = 3,
    NonzeroExit = 4,
    Cancelled = 5,
};

struct ProcessRequest {
    Path executable;
    std::vector<std::string> arguments;
    std::chrono::milliseconds timeout {10000};
    size_t stdout_limit_bytes {1024 * 1024};
    size_t stderr_limit_bytes {4096};
    std::stop_token stop_token;
};

struct ProcessResult {
    ProcessExitState state {ProcessExitState::FailedStart};
    i32 exit_code {0};
    std::string standard_output;
    std::string standard_error;
    std::string error_message;

    [[nodiscard]] auto ok() const noexcept -> bool;
};

using ProcessExecutor = std::function<ProcessResult(const ProcessRequest&);>;

[[nodiscard]] auto run_process(const ProcessRequest& request) -> ProcessResult;
[[nodiscard]] auto process_exit_state_name(ProcessExitState state) -> std::string;
```

This is the `ProcessRunner` port: production code calls `run_process` and tests/adapters inject a `ProcessExecutor`. `ProcessResult` is returned for every attempted invocation. `exit_code` is meaningful only for `Success` and `NonzeroExit`. Output is captured to the supplied byte limits. No implementation may invoke a shell.

Cancellation uses the C++20 standard library. A caller owns `std::stop_source` and places its token in each `ProcessRequest`. A stop requested before launch returns `Cancelled` without a child. A stop requested while waiting terminates and reaps the active process before returning `Cancelled`. Cancellation is idempotent and never rewritten as `TimedOut`, `Crashed`, or `NonzeroExit`.

ProbeService

```
struct ProbeResult {
    bool success {false};
    VideoMetadata metadata;
    ProcessResult process;
    std::string error_message;

    [[nodiscard]] auto ok() const noexcept -> bool;
};

class ProbeService {
public:
    explicit ProbeService(ProcessExecutor executor = run_process);

    [[nodiscard]] auto probe(
        const Path& executable,
        const Path& source_path,
        std::stop_token stop_token = {}) const -> ProbeResult;
};
```

ProbeService owns ffmpeg invocation and JSON-to-domain translation. It forwards the stop token through ProcessRequest. ProbeResult preserves both parsed metadata and bounded process evidence, so the CLI and Qt adapter surface the same diagnostic. ProbeCoordinator is the Qt compatibility adapter: it runs ProbeService::probe off the UI thread and queues exactly one finished signal.

Planner

```
struct OutputPlanInput {
    VideoMetadata metadata;
    std::optional<Path> chapter_source_path;
    bool uses_embedded_chapters {false};
    std::vector<ChapterSegment> chapters;
    ExportSettings settings;
    EncoderEnvironment environment;
};

struct OutputPlanResult {
    std::vector<ResolvedExportJob> jobs;
    std::vector<std::string> errors;

    [[nodiscard]] auto ok() const noexcept -> bool;
};

[[nodiscard]] auto plan_outputs(
    const std::vector<OutputPlanInput>& inputs) -> OutputPlanResult;
```

The roadmap calls this responsibility the planner; the concrete incremental API remains plan_outputs. It is pure and deterministic: it performs no process launch or filesystem write. Each ResolvedExportJob contains final output paths and exact ffmpeg argument vectors, so dry run and real export consume the same plan.

ChapterSource resolution happens before planning. A ChapterFile parse failure or unavailable embedded chapter set is reported through the existing resolver/config error vector. A valid OutputPlanResult contains only Jobs with at least one valid Chapter and probed metadata. Invalid inputs never reach export.

Export engine and progress

```
enum class ExportExitCode : u8 {
    Success = 0,
    ExportFailure = 2,
    ToolingError = 3,
};

struct RenderedSegment {
    Path source_path;
    u16 chapter_index {0};
    std::string chapter_name;
    Path output_path;
    ProcessResult process;
    bool skipped {false};
    bool overwrote_existing {false};

    [[nodiscard]] auto ok() const noexcept -> bool;
};
```

Execution retains the existing options, partial-result aggregate, callbacks, and injected process executor. Cancellation is the single additive field required by the shared boundary:

```
struct ExportRunOptions {
    std::chrono::milliseconds process_timeout {std::chrono::hours {24}};
    size_t stdout_limit_bytes {1024 * 1024};
    size_t stderr_limit_bytes {4096};
    std::stop_token stop_token;
    std::function<void(
        size_t,
        size_t,
        size_t,
        size_t,
        const ResolvedExportJob&,
        const PlannedExportSegment&> segment_started;
    std::function<void(const RenderedSegment&> segment_finished;
    std::function<void(const std::string&> message;
};

struct ExportRunResult {
    std::vector<ExportJobResult> jobs;
    ExportExitCode exit_code {ExportExitCode::Success};
    bool stopped_early {false};

    [[nodiscard]] auto ok() const noexcept -> bool;
};

class ExportRunner final {
public:
    explicit ExportRunner(ProcessExecutor executor = run_process);

    [[nodiscard]] auto run(
        const std::vector<ResolvedExportJob>& jobs,
        const ExportRunOptions& options = {}) const -> ExportRunResult;
};
```

The roadmap calls this responsibility the `ExportEngine`; `ExportRunner` is its compatibility-preserving concrete API until the owning migration chooses a final file/type name. The only required additive contract is `std::stop_token` in `ExportRunOptions`, forwarded into every process request. Existing Job/result aggregates and callbacks remain valid during the move.

`run` always returns `ExportRunResult` so completed segments survive a later failure or cancellation. Callbacks are synchronous on the engine thread. `segment_started` precedes its matching `segment_finished`; no later segment starts after cancellation; messages and counts preserve their current ordering. Adapters MUST copy callback data before queueing it and MUST NOT call UI objects from the engine thread.

Manifests

```
struct ManifestWriteResult {
    bool success {false};
    std::vector<Path> written_paths;
    std::vector<Path> preserved_media_paths;
    std::vector<std::string> errors;
    std::vector<ManifestJobWriteResult> jobs;

    [[nodiscard]] auto ok() const noexcept -> bool;
};

[[nodiscard]] auto write_manifests(
    const std::vector<ResolvedExportJob>& planned_jobs,
    const ExportRunResult& run_result,
    const std::optional<Path>& aggregate_json_path = std::nullopt,
    const std::optional<Path>& aggregate_csv_path = std::nullopt)
-> ManifestWriteResult;
```

`write_manifests` receives the immutable plan and partial result after success, failure, or cancellation. Schema version, source identity, planned path, process state, skipped/overwrite flags, and cancellation MUST be serialized. A manifest failure never deletes or rewrites media; it returns `success == false`, preserves media paths, and adds the intended path and diagnostic to `errors`.

ChapterSource to RenderedSegment



The same `OutputPlanResult.jobs` feed dry-run display and execution. The CLI and Qt adapter **MUST NOT** reconstruct paths, commands, or Chapter boundaries after planning.

Cancellation and Partial Work

Cancellation point	Required result	Artifact rule
Before process launch	<code>ProcessExitState::Cancelled</code>	No child process or output is created.
Between segments	<code>ExportRunResult.stopped_early == true</code>	No later segment starts; completed outputs remain.
During ffprobe	<code>ProbeResult.process.state == Cancelled</code>	Active child is terminated and reaped.
During ffmpeg	RenderedSegment process state Cancelled	Partial file follows existing output policy.
After media, before manifest	<code>ExportRunResult</code> preserved	Manifest records completed and cancelled states.

The caller owns `std::stop_source`. Services forward its token in each `ProcessRequest` and observe it between segments. Cancellation produces one terminal result and is never rewritten as timeout, crash, or generic export failure.

Failure and Exit-Code Mapping

The service layer keeps detailed errors; presentation adapters decide how to display them.

Failure	Concrete result state	CLI exit	GUI presentation
Usage, settings, ChapterFile, or chapter validation	Resolver/planner errors; no export	1	Validation message; no work starts
ffprobe missing, start, timeout, crash, or nonzero	ProbeResult.success == false plus ProcessExitState	3	Tooling error with source and diagnostic
ffmpeg cannot start	ProcessExitState::FailedStart	3	Tooling error; preserve prior outputs
ffmpeg timeout, crash, or nonzero	Matching ProcessExitState	2	Export failure with chapter and stderr
User cancellation	ProcessExitState::Cancelled and stopped early	2	Distinct cancelled state, not success
Output filesystem failure	ExportJobResult.error_message	2	Export failure with intended path
Requested manifest cannot be written	ManifestWriteResult.success == false	2	Media remains; manifest failure is visible
Every requested segment and manifest succeeds	ExportExitCode::Success	0	Completed

Exit values remain 0 success, 1 usage/validation, 2 export/manifest/cancellation, and 3 tooling/probe/start failure. The stable migration introduces no new public exit code. A Batch with any failed or cancelled Job cannot return 0.

Synchronous Engine, Asynchronous Qt

The shared API is deliberately blocking. The CLI owns the calling thread and invokes it directly. The Qt adapter owns one worker execution per Batch:

- 1 Copy standard C++ request values before leaving the UI thread.
- 2 Queue work to a worker that owns or borrows the shared services for that run.
- 3 Copy callback data and terminal results into immutable Qt value payloads.
- 4 Deliver those payloads through queued connections to the UI thread.
- 5 Make the cancel slot call only `std::stop_source::request_stop()`.
- 6 Emit exactly one terminal signal, then release the worker and source.

The adapter **MUST NOT** call `ExportRunner::run` on the UI thread. Export callbacks **MUST NOT** dereference `QWidget` or other UI-owned `QObject` instances. Qt types do not appear in shared headers, and the engine never owns a `QObject`, `QThread`, `QProcess`, `QString`, or `QJson` value.

Windows Process Boundary

The Windows implementation of `run_process` is the initial concrete process service. It MUST:

- call `CreateProcessW` directly with wide executable and argument paths;
- use the repository's tested Windows quoting rules and never invoke `cmd . exe` or another shell;
- redirect stdout and stderr through bounded pipes and continue draining while waiting;
- use `CREATE_NO_WINDOW` for CLI and background desktop execution;
- distinguish start failure, timeout, cancellation, crash, and nonzero exit;
- terminate and reap the active child on timeout/cancellation, close every handle, and then return;
- keep `windows . h`, handles, and quoting helpers out of public service headers.

Portable tests inject a fake `ProcessExecutor`. A future non-Windows implementation MAY use another process API without changing the service contracts, error mapping, CLI codes, or golden command vectors.

Migration Sequence and Gates

Each phase is one focused Linear task and PR. A phase begins only after the previous gate is green.

Phase	Change	Behavior gate
0	Freeze current GUI/CLI fixtures and target contracts	Golden help, exit-code, argv, naming, manifest, settings, and progress tests pass
1	Move process/probe into src/services using <code>std::stop_token</code>	Existing CLI process/probe tests plus paths-with-spaces and cancellation pass
2	Add Qt ProbeCoordinator compatibility adapter	Qt owner-thread delivery, responsiveness, cancellation, and destruction tests pass
3	Move ChapterSource resolution, planning, export, and manifests	Dry run and real export consume identical plans; JSON/CSV goldens are byte-stable
4	Rewire CLI to the shared engine	CLI end-to-end fixtures preserve output, diagnostics, and exact exit codes
5	Add the Qt worker adapter and remove duplicated QProcess workflow	Qt signals are ordered, UI stays responsive, cancel emits one terminal result
6	Remove obsolete orchestration and finalize CMake targets	Full Windows CI, slow ffmpeg, packaged GUI, and packaged CLI smoke tests pass

Every phase compares its branch against main, updates the [repo map](#) (`../knowledge/architecture/repo-map.md`) when ownership moves, and records confirmed checks only. A migration may add adapters around current names (`ChapterSegment`, `BatchJob`, and `ResolvedExportJob`) before final renames; it must not combine a semantic move with an unrelated rename.

Repository and Developer Map

Read or change	Canonical location
Domain vocabulary	CONTEXT.md (./CONTEXT.md)
Accepted shared-engine decision	ADR 0001 (./knowledge/architecture/decisions/0001-shared-qt-free-engine.md)
Current and transitional directory ownership	repo map (./knowledge/architecture/repo-map.md)
C++ and Qt conventions	CODING_STYLE.md (./CODING_STYLE.md)
Verification tiers and remediation	verification guide (verification.md)
Architecture documentation index	knowledge base (./knowledge/README.md)
Historical CLI plan	historical companion (vidchopper_cli_architecture_plan.md)
Core domain implementation	src/core/
Current CLI workflow to migrate	src/cli/
Current Qt workflow to replace	src/qt/services/
Native, integration, and Qt tests	tests/

The Markdown file is authoritative. Its tracked PDF is regenerated by `docs/scripts/render-guide-pdf.py`. `npm test` in `docs/` verifies required sections, cross-links, public-contract tokens, PDF source digest, page bounds, and CI routing.

Non-Goals

- A general-purpose video editor, timeline editor, or plugin framework.
- An interactive shell or long-running command server before a concrete workflow requires one.
- A Qt dependency hidden behind typedefs, PIMPL, callbacks, JSON, or generated headers.
- Concurrent ffmpeg exports in the stable migration; sequential execution remains the baseline.
- Changing persisted settings, naming, command ordering, manifests, or exit codes as refactor cleanup.
- Shipping ffmpeg/ffprobe inside the shared engine or changing their distribution policy.
- A cross-platform process implementation in the first migration; the boundary enables it later.
- Runtime code changes in VID-38. This issue publishes the contract and its deterministic artifact.

Stable-Release Completion Contract

The shared-engine roadmap reaches this guide's boundary when both executables construct the same Qt-free services; golden behavior and packaged smoke tests are green; the GUI contains only presentation adapters; the CLI contains only CLI adapters; and no Qt or Win32 type crosses a shared public header. Until those conditions are met, this guide is the implementation target rather than a claim that migration is already complete.